



Basic to Advanced Fundamentals

50 Problems · Clear Explanations · Step-by-Step Solutions
Arrays · Linked Lists · Stacks · Queues · Trees · Graphs
Sorting · Searching · Hashing · DP · Greedy · Advanced Topics

TABLE OF CONTENTS

Chapter 1 — Introduction to DSA

- › What is DSA?
- › Big-O Notation
- › Time & Space Complexity

Chapter 2 — Arrays

- › 1D & 2D Arrays
- › Traversal, Insertion, Deletion
- › Problems: Two Sum, Max Subarray

Chapter 3 — Linked Lists

- › Singly & Doubly Linked List
- › Insertion, Deletion, Reversal
- › Problems: Detect Cycle, Merge Lists

Chapter 4 — Stacks & Queues

- › Stack Operations (LIFO)
- › Queue Operations (FIFO)
- › Problems: Valid Parentheses, BFS

Chapter 5 — Trees

- › Binary Tree, BST, AVL
- › Inorder/Preorder/Postorder
- › Problems: Height, LCA, Level Order

Chapter 6 — Heaps

- › Min-Heap & Max-Heap
- › Heap Operations
- › Problems: Kth Largest, Median Stream

Chapter 7 — Hashing

- › Hash Map & Hash Set
- › Collision Resolution
- › Problems: Anagram, Frequency Count

Chapter 8 — Graphs

- › BFS, DFS, Adjacency List
- › Directed & Undirected
- › Problems: Shortest Path, Cycle Detection

Chapter 9 — Sorting Algorithms

- › Bubble, Selection, Insertion
- › Merge Sort, Quick Sort
- › Comparison & Complexity Table

Chapter 10 — Searching Algorithms

- › Linear Search, Binary Search
- › Recursive Binary Search
- › Problems: Search in Rotated Array

Chapter 11 — Dynamic Programming

- › Memoization vs Tabulation
- › Classic DP Patterns
- › Problems: Fibonacci, 0-1 Knapsack, LCS

Chapter 12 — Greedy Algorithms

- › Greedy Choice Property
- › Activity Selection

- › Problems: Coin Change, Fractional Knapsack

Chapter 13 — Advanced Topics

- › Trie, Segment Tree overview
- › Sliding Window, Two Pointers
- › Problems: Longest Prefix, Range Sum

CHAPTER
1

Introduction to DSA

Why Every Programmer Needs Data Structures & Algorithms

1.1 What is Data Structure?

A **Data Structure** is a way of organising and storing data in a computer so that it can be accessed and modified efficiently. Think of it like a bookshelf — you can organise books by genre, author, or title so you can find any book quickly.

- **Linear DS:** Arrays, Linked Lists, Stacks, Queues (elements in sequence)
- **Non-Linear DS:** Trees, Graphs (hierarchical or network relationships)
- **Hash-Based DS:** Hash Maps, Hash Sets (key-value lookup in $O(1)$)

1.2 What is an Algorithm?

An **Algorithm** is a step-by-step procedure to solve a problem. A good algorithm is correct, efficient, and easy to understand. The same problem can be solved by many algorithms — we choose the best one using **complexity analysis**.

1.3 Big-O Notation — Measuring Efficiency

Big-O describes the **worst-case growth rate** of time or space as input size **n** grows. It allows us to compare algorithms independently of hardware.

Big-O	Name	Example	n=100 ops
$O(1)$	Constant	Array index access	1
$O(\log n)$	Logarithmic	Binary Search	7
$O(n)$	Linear	Linear Search	100
$O(n \log n)$	Log-Linear	Merge Sort	700
$O(n^2)$	Quadratic	Bubble Sort	10,000
$O(2^n)$	Exponential	Subset enumeration	2^{100}

■ **Tip:** Always aim for $O(n \log n)$ or better for large inputs. $O(n^2)$ becomes too slow beyond ~10,000 elements.

CHAPTER
2

Arrays

The Most Fundamental Linear Data Structure

2.1 What is an Array?

An **Array** stores elements at **contiguous memory locations**. Each element is accessed using its **index** (0-based in Python). Arrays are fast to read ($O(1)$) but slow to insert/delete in the middle ($O(n)$).

```
# Python array basics
arr = [10, 20, 30, 40, 50]
print(arr[0]) # 10 - O(1) access
arr.append(60) # O(1) amortised
arr.insert(2, 25) # O(n) - shifts elements
arr.pop(1) # O(n) - removes index 1
# 2D Array
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(matrix[1][2]) # 6
```

	Access $O(1)$ 	
Time Complexity:	Search $O(n)$	Space Complexity: $O(n)$

■ Problem 1 (Easy): Two Sum

Given an array of integers **nums** and a target integer **target**, return the **indices** of the two numbers such that they add up to target. Example: `nums = [2, 7, 11, 15]`, `target = 9` → Output: `[0, 1]` (because `2 + 7 = 9`)

■ Solution & Explanation

Approach: Use a **Hash Map** to store each number and its index as we iterate. For each number, check if `(target - number)` already exists in the map. **Step-by-step:** 1. Create an empty dictionary: `seen = {}` 2. Loop through `nums` with index `i` and value `num` 3. Calculate `complement = target - num` 4. If `complement` is in `seen` → return `[seen[complement], i]` 5. Else, store `seen[num] = i` and continue

```
def two_sum(nums, target):
    seen = {} # stores {value: index}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i] # found!
        seen[num] = i
    return [] # no solution
# Test
print(two_sum([2, 7, 11, 15], 9)) # [0, 1]
print(two_sum([3, 2, 4], 6)) # [1, 2]
```

Time Complexity:	$O(n)$	Space Complexity:	$O(n)$
-------------------------	--------------------------	--------------------------	--------------------------

■ Problem 2 (Medium): Maximum Subarray (Kadane's Algorithm)

Find the contiguous subarray with the largest sum. Example: `nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]` → Output: 6 (subarray `[4, -1, 2, 1]`)

■ Solution & Explanation

Approach (Kadane's Algorithm): Scan once. At each position decide: extend the current subarray OR start fresh from the current element — whichever is larger. **Key idea:** If `current_sum + num < num`, discard the previous subarray and start from `num`.

```
def max_subarray(nums): current_sum = max_sum = nums[0] for num in nums[1:]:  
    current_sum = max(num, current_sum + num) max_sum = max(max_sum, current_sum) return  
max_sum print(max_subarray([-2,1,-3,4,-1,2,1,-5,4])) # 6
```

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

CHAPTER
3

Linked Lists

Dynamic Data Storage with Pointers

3.1 Singly Linked List

A **Linked List** is a sequence of **Nodes** where each node stores data and a pointer (**next**) to the next node. Unlike arrays, nodes are **not contiguous in memory** — they can be anywhere, connected via pointers. Insertion/Deletion at head is $O(1)$ but search is $O(n)$.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None # pointer to next node
class LinkedList:
    def __init__(self):
        self.head = None
    def insert_front(self, data):
        # O(1)
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
    def display(self):
        cur = self.head
        while cur:
            print(cur.data, end=' -> ')
            cur = cur.next
        print('None')
```

■ Problem 3 (Easy): Reverse a Linked List

Given the head of a singly linked list, reverse it and return the new head. Example: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$

■ Solution & Explanation

Approach (Iterative): Use three pointers: prev, current, next_node. On each step, flip the arrow and advance. **Visualisation:** Initially: $\text{None} \leftarrow [\text{prev}]$ $[\text{cur}] \rightarrow 2 \rightarrow 3 \rightarrow \text{None}$ After step1: $\text{None} \leftarrow 1 \leftarrow [\text{prev}]$ $[\text{cur}] \rightarrow 3 \rightarrow \text{None}$ Continue until cur becomes None, return prev

```
def reverse_list(head):
    prev = None
    current = head
    while current:
        next_node = current.next
        current.next = prev # flip arrow
        prev = current # advance prev
        current = next_node # advance current
    return prev # new head
# Result: 5→4→3→2→1→None
```

Time Complexity: **$O(n)$** **Space Complexity:** **$O(1)$**

■ Problem 4 (Medium): Detect a Cycle (Floyd's Algorithm)

Given a linked list, determine if it contains a cycle. Example: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 2$ (cycle) $\rightarrow$ True

■ Solution & Explanation

Approach (Two Pointer / Tortoise and Hare): Use two pointers: **slow** (moves 1 step) and **fast** (moves 2 steps). If they ever meet, a cycle exists. If fast reaches None, no cycle. **Why it works:** In a cycle, fast gains on slow by 1 step each iteration, so they must eventually meet — like two runners on a circular track.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next # 1 step
        fast = fast.next.next # 2 steps
    if slow == fast: # they met!
        return True
    return False
```

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

**CHAPTER
4**

Stacks & Queues

LIFO and FIFO Data Structures

4.1 Stack (Last In, First Out)

A **Stack** works like a pile of plates: you always add/remove from the **top**. Operations: **push** (add), **pop** (remove top), **peek** (see top). Used in: undo/redo, function call stack, expression evaluation.

```
# Python list as stack
stack = []
stack.append(10) # push
stack.append(20)
stack.append(30)
print(stack.pop()) # 30 - last in, first out
print(stack[-1]) # peek: 20
```

■ Problem 5 (Easy): Valid Parentheses

Given a string containing '(', ')', '{', '}', '[', ']', determine if it is valid. Rules: Open brackets must be closed in the correct order. Examples: '()[]{}' → True, '(]' → False, '([)]' → False

■ Solution & Explanation

Approach: Use a stack. For each opening bracket, push it. For each closing bracket, check if it matches the top of the stack. Matching pairs:) pairs with (,] with [, } with {

```
def is_valid(s):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'
    for ch in s:
        if ch in '([{':
            stack.append(ch)
        elif ch in ')]}':
            if not stack or stack[-1] != pairs[ch]:
                return False
            stack.pop()
    return len(stack) == 0
print(is_valid('()[]{}')) # True
print(is_valid('()])')) # False
```

Time Complexity: $O(n)$ **Space Complexity:** $O(n)$

4.2 Queue (First In, First Out)

A **Queue** works like a line at a ticket counter: elements are added at the **rear** and removed from the **front**. Used in: BFS, task scheduling, print queues.

```
from collections import deque
q = deque()
q.append(1) # enqueue
q.append(2)
q.append(3)
print(q.popleft()) # dequeue → 1 (first in, first out)
```

■ Problem 6 (Medium): BFS Level Order Traversal

Given the root of a binary tree, return node values level by level (left to right). Example: Tree $1 \rightarrow [2,3] \rightarrow [4,5,6] \rightarrow [[1],[2,3],[4,5,6]]$

■ Solution & Explanation

Approach: Use a Queue (BFS). Start with root, process all nodes at each level before moving to the next.

```
from collections import deque
def level_order(root):
    if not root:
        return []
    result, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            # process current level node
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result
```

Time Complexity: $O(n)$ **Space Complexity:** $O(n)$

CHAPTER
5

Trees

Hierarchical Data — Binary Trees, BST & Traversals

5.1 Tree Terminology

- **Root:** Top-most node with no parent
- **Leaf:** Node with no children
- **Height:** Longest path from root to a leaf
- **BST Property:** left child < parent < right child

5.2 Tree Node & Traversals

```
class TreeNode: def __init__(self, val): self.val = val self.left = self.right = None #
Three DFS Traversals def inorder(root): # Left → Root → Right (sorted for BST) if not
root: return [] return inorder(root.left) + [root.val] + inorder(root.right) def
preorder(root): # Root → Left → Right if not root: return [] return [root.val] +
preorder(root.left) + preorder(root.right) def postorder(root): # Left → Right → Root
if not root: return [] return postorder(root.left) + postorder(root.right) + [root.val]
```

■ Problem 7 (Easy): Maximum Depth of Binary Tree

Find the maximum depth (height) of a binary tree. Example: 3 / \ 9 20 / \ 15 7 Output: 3

■ Solution & Explanation

Approach (Recursion): The height of a tree is $1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$. Base case: empty node returns 0. Think of it like: ask each subtree for its height, take the bigger one, add 1 for the current node.

```
def max_depth(root): if not root: return 0 # base case left_h = max_depth(root.left)
right_h = max_depth(root.right) return 1 + max(left_h, right_h) # For tree above:
max_depth → 3
```

Time Complexity: **$O(n)$** **Space Complexity:** **$O(h)$ where h
= height**

■ Problem 8 (Medium): Lowest Common Ancestor (LCA)

Given a BST and two nodes p, q , find their Lowest Common Ancestor. LCA = the deepest node that is an ancestor of both p and q .

■ Solution & Explanation

BST Approach: Exploit the BST property! • If both p and q are less than root → LCA is in LEFT subtree • If both p and q are greater than root → LCA is in RIGHT subtree • Otherwise → current root IS the LCA

```
def lca_bst(root, p, q): while root: if p.val < root.val and q.val < root.val: root =  
    root.left # both in left elif p.val > root.val and q.val > root.val: root = root.right  
    # both in right else: return root # split point = LCA return None
```

	$O(h)$ — $O(\log n)$ for	
Time Complexity:	balanced BST	Space Complexity: $O(1)$

CHAPTER
6

Heaps

Priority Queues — Min-Heap & Max-Heap

6.1 What is a Heap?

A **Heap** is a **complete binary tree** satisfying the **heap property**: in a **Min-Heap** every parent is $\leq$ its children (minimum is always at root). In a **Max-Heap** every parent is $\geq$ its children. Used in priority queues, Dijkstra's algorithm, heap sort.

```
import heapq # Min-Heap (Python default) heap = [] heapq.heappush(heap, 5)
heapq.heappush(heap, 1) heapq.heappush(heap, 3) print(heapq.heappop(heap)) # 1
(minimum) # Max-Heap: negate values max_heap = [] heapq.heappush(max_heap, -10)
heapq.heappush(max_heap, -5) print(-heapq.heappop(max_heap)) # 10 (maximum)
```

■ Problem 9 (Medium): Kth Largest Element

Find the kth largest element in an unsorted array. Example: nums = [3,2,1,5,6,4], k=2 → Output: 5

■ Solution & Explanation

Approach: Maintain a **Min-Heap of size k**. After processing all elements, the root of the heap (minimum of the heap) is the kth largest. **Why?** If heap has k elements and all are the k largest we've seen, the smallest among them = kth largest overall.

```
import heapq def find_kth_largest(nums, k): heap = [] for num in nums:
heapq.heappush(heap, num) if len(heap) > k: heapq.heappop(heap) # remove smallest
return heap[0] # kth largest print(find_kth_largest([3,2,1,5,6,4], 2)) # 5
```

Time Complexity: $O(n \log k)$ **Space Complexity:** $O(k)$

CHAPTER
7

Hashing

O(1) Lookups with Hash Maps & Hash Sets

7.1 How Hashing Works

A **Hash Map** converts a key into an array index using a **hash function**. This enables average O(1) insert, delete, and lookup. Python's dict and set are hash-based.

```
# Hash Map (dict) freq = {} words =
['apple', 'banana', 'apple', 'cherry', 'banana', 'apple'] for w in words: freq[w] =
freq.get(w, 0) + 1 print(freq) # {'apple':3, 'banana':2, 'cherry':1} # Hash Set (set) -
for fast membership testing seen = set() seen.add(10) print(10 in seen) # True - O(1)
```

■ Problem 10 (Easy): Valid Anagram

Given two strings s and t, return True if t is an anagram of s. Anagram = same characters, same frequencies, different order. Example: s='anagram', t='nagaram' → True; s='rat', t='car' → False

■ Solution & Explanation

Approach: Count character frequencies for both strings and compare. If counts are identical → anagram.

```
from collections import Counter def is_anagram(s, t): return Counter(s) == Counter(t) #
Manual approach (for understanding): def is_anagram_manual(s, t): if len(s) != len(t):
return False count = {} for c in s: count[c] = count.get(c,0) + 1 for c in t: if c not
in count or count[c] == 0: return False count[c] -= 1 return True
```

Time Complexity:**O(n)****Space Complexity:****O(k) where k =
unique chars**

■ Problem 11 (Medium): Longest Consecutive Sequence

Given unsorted array, find length of longest consecutive sequence. Example: nums = [100,4,200,1,3,2] → Output: 4 (sequence: 1,2,3,4)

■ Solution & Explanation

Approach: Put all numbers in a **set**. For each number n, if (n-1) is NOT in the set, it's the start of a sequence. Then count how far the sequence extends.

```
def longest_consecutive(nums): num_set = set(nums) best = 0 for n in num_set: if n - 1
not in num_set: # start of sequence cur, length = n, 1 while cur + 1 in num_set: cur +=
1; length += 1 best = max(best, length) return best
print(longest_consecutive([100,4,200,1,3,2])) # 4
```

Time Complexity:**O(n)****Space Complexity:****O(n)**

CHAPTER
8

Graphs

BFS, DFS, Shortest Path & Cycle Detection

8.1 Graph Representation

A **Graph** is a set of **vertices (nodes)** connected by **edges**. Can be **directed** (edges have direction) or **undirected**. Two main representations:

- **Adjacency List:** Dict/list of neighbors — space efficient $O(V+E)$
- **Adjacency Matrix:** 2D grid — $O(1)$ edge lookup but $O(V^2)$ space

```
# Adjacency List representation graph = { 0: [1, 2], 1: [0, 3, 4], 2: [0], 3: [1], 4: [1] }
# BFS - Level by level exploration from collections import deque
def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order
```

■ Problem 12 (Medium): Number of Islands

Given a 2D grid of '1' (land) and '0' (water), count the number of islands. An island is surrounded by water and formed by connecting adjacent land (up, down, left, right). Example: grid with 3 separate groups of '1' → 3

■ Solution & Explanation

Approach (DFS Flood Fill): Scan each cell. When we find a '1' (land), do DFS to mark the entire island as visited (change to '0'). Count each DFS call as one island.

```
def num_islands(grid):
    if not grid: return 0
    count = 0
    def dfs(r, c):
        if r < 0 or r >= len(grid) or c < 0 or c >= len(grid[0]): return
        if grid[r][c] != '1': return
        grid[r][c] = '0' # mark as visited
        dfs(r+1, c)
        dfs(r-1, c)
        dfs(r, c+1)
        dfs(r, c-1)
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == '1':
                dfs(r, c)
                count += 1
    return count
```

Time Complexity: $O(m \times n)$

Space Complexity:

 $O(m \times n)$ —
recursion
stack

CHAPTER
9

Sorting Algorithms

From Bubble Sort to Merge Sort & Quick Sort

9.1 Sorting Overview

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

9.2 Merge Sort — Divide and Conquer

```
def merge_sort(arr): if len(arr) <= 1: return arr mid = len(arr) // 2 left =
merge_sort(arr[:mid]) right = merge_sort(arr[mid:]) return merge(left, right) def
merge(left, right): result = []; i = j = 0 while i < len(left) and j < len(right): if
left[i] <= right[j]: result.append(left[i]); i += 1 else: result.append(right[j]); j +=
1 result.extend(left[i:]); result.extend(right[j:]) return result
print(merge_sort([38,27,43,3,9,82])) # [3,9,27,38,43,82]
```

Time Complexity: $O(n \log n)$ **always** **Space Complexity:** $O(n)$

■ Problem 13 (Medium): Sort Colors (Dutch Flag Problem)

Given array with 0s, 1s, 2s, sort them in-place without using built-in sort. Example: [2,0,2,1,1,0] → [0,0,1,1,2,2]

■ Solution & Explanation

Approach (Three Pointers): Use low, mid, high pointers. • $\text{nums}[\text{mid}] == 0 \rightarrow$ swap with low, advance both • $\text{nums}[\text{mid}] == 1 \rightarrow$ advance mid • $\text{nums}[\text{mid}] == 2 \rightarrow$ swap with high, decrease high

```
def sort_colors(nums): low = mid = 0; high = len(nums)-1 while mid <= high: if
nums[mid] == 0: nums[low], nums[mid] = nums[mid], nums[low] low += 1; mid += 1 elif
nums[mid] == 1: mid += 1 else: nums[mid], nums[high] = nums[high], nums[mid] high -= 1
return nums print(sort_colors([2,0,2,1,1,0])) # [0,0,1,1,2,2]
```

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

CHAPTER
10

Searching Algorithms

Linear Search, Binary Search & Variants

10.1 Binary Search

Binary Search works on **sorted arrays**. It repeatedly halves the search space by comparing the middle element with the target. Each step eliminates half the remaining elements.

```
def binary_search(arr, target): left, right = 0, len(arr) - 1 while left <= right: mid = (left + right) // 2 if arr[mid] == target: return mid # found at index mid elif arr[mid] < target: left = mid + 1 # search right half else: right = mid - 1 # search left half return -1 # not found arr = [1,3,5,7,9,11,13,15] print(binary_search(arr, 7)) # 3 (index) print(binary_search(arr, 6)) # -1 (not found)
```

Time Complexity: $O(\log n)$ **Space Complexity:** $O(1)$

■ Problem 14 (Medium): Search in Rotated Sorted Array

A sorted array was rotated at some pivot. Find the index of target. Example: nums = [4,5,6,7,0,1,2], target=0 → Output: 4

■ Solution & Explanation

Approach: Modified Binary Search. At each step, one half is always sorted. Determine which half is sorted, then check if target falls in that range.

```
def search_rotated(nums, target): left, right = 0, len(nums)-1 while left <= right: mid = (left + right) // 2 if nums[mid] == target: return mid # Left half is sorted if nums[left] <= nums[mid]: if nums[left] <= target < nums[mid]: right = mid - 1 else: left = mid + 1 # Right half is sorted else: if nums[mid] < target <= nums[right]: left = mid + 1 else: right = mid - 1 return -1 print(search_rotated([4,5,6,7,0,1,2], 0)) # 4
```

Time Complexity: $O(\log n)$ **Space Complexity:** $O(1)$

CHAPTER
11

Dynamic Programming

Solve Complex Problems by Breaking Them into Subproblems

11.1 What is Dynamic Programming?

Dynamic Programming (DP) solves problems by breaking them into **overlapping subproblems** and storing results to avoid recomputation (**memoization**). Two approaches:

- **Top-Down (Memoization):** Recursive + cache results
- **Bottom-Up (Tabulation):** Iterative, fill table from base cases

■ Problem 15 (Easy): Fibonacci Number

Find the nth Fibonacci number. $F(0)=0$, $F(1)=1$, $F(n) = F(n-1) + F(n-2)$ Example: $\text{fib}(10) \rightarrow 55$

■ Solution & Explanation

Naïve recursion repeats the same subproblems exponentially. DP stores results so each is solved once.

```
# Top-Down DP (Memoization)
def fib_memo(n, memo={}):
    if n <= 1: return n
    if n in memo: return memo[n] # cached!
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]
# Bottom-Up DP (Tabulation)
def fib_dp(n):
    if n <= 1: return n
    dp = [0] * (n+1)
    dp[1] = 1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
print(fib_dp(10)) # 55
```

Time Complexity:

 $O(n)$

Space Complexity:

$O(n)$ —
optimisable to
 $O(1)$

■ Problem 16 (Hard): 0-1 Knapsack Problem

Given n items with weights and values, and a bag capacity W, find the maximum value you can carry. Each item can be taken at most once. $\text{weights}=[2,3,4,5]$, $\text{values}=[3,4,5,6]$, $W=8 \rightarrow \text{Output: } 10$

■ Solution & Explanation

Approach: Build a 2D DP table. $\text{dp}[i][w]$ = max value using first i items with capacity w. **Recurrence:** • If $\text{weight}[i] > w$: $\text{dp}[i][w] = \text{dp}[i-1][w]$ (can't include item) • Else: $\text{dp}[i][w] = \max(\text{dp}[i-1][w], \text{dp}[i-1][w-\text{weight}[i]] + \text{value}[i])$

```
def knapsack(weights, values, W):
    n = len(weights)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            # Don't take item i
            dp[i][w] = dp[i-1][w]
            # Take item i (if it fits)
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w], dp[i-1][w-weights[i-1]] + values[i-1])
    return dp[n][W]
print(knapsack([2,3,4,5], [3,4,5,6], 8)) # 10
```

Time Complexity:

 $O(n \times W)$

Space Complexity:

 $O(n \times W)$

■ Problem 17 (Hard): Longest Common Subsequence (LCS)

Find the length of the longest common subsequence between two strings. Example: $s1='ABCBDAB'$, $s2='BDCAB' \rightarrow LCS='BCAB' \rightarrow \text{Length}=4$

■ Solution & Explanation

Approach: $dp[i][j]$ = LCS of $s1[:i]$ and $s2[:j]$ • If $s1[i-1]==s2[j-1]$: $dp[i][j] = dp[i-1][j-1] + 1$ • Else: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

```
def lcs(s1, s2): m, n = len(s1), len(s2) dp = [[0]*(n+1) for _ in range(m+1)] for i in range(1, m+1): for j in range(1, n+1): if s1[i-1] == s2[j-1]: dp[i][j] = dp[i-1][j-1] + 1 else: dp[i][j] = max(dp[i-1][j], dp[i][j-1]) return dp[m][n]
print(lcs('ABCBDAB', 'BDCAB')) # 4
```

Time Complexity: $O(m \times n)$ **Space Complexity:** $O(m \times n)$

CHAPTER
12

Greedy Algorithms

Make the Best Local Choice at Every Step

12.1 Greedy Strategy

A **Greedy Algorithm** makes the **locally optimal choice** at each step, hoping it leads to the globally optimal solution. Works when the problem has the **Greedy Choice Property** and **Optimal Substructure**.
Simpler than DP — no backtracking needed.

■ Problem 18 (Easy): Coin Change (Greedy for special cases)

Given coin denominations and a target amount, find the minimum number of coins. coins=[1,5,10,25], amount=41 → Output: 4 coins (25+10+5+1)

■ Solution & Explanation

Note: Greedy works for standard coin systems (like US coins). For arbitrary denominations, use DP.
Greedy approach: Always pick the largest coin that fits the remaining amount.

```
def coin_change_greedy(coins, amount): coins.sort(reverse=True) # largest first count = 0
for coin in coins: while amount >= coin: amount -= coin count += 1 return count if amount == 0 else -1
# DP version (works for all coin sets):
def coin_change_dp(coins, amount): dp = [float('inf')] * (amount+1) dp[0] = 0
for a in range(1, amount+1): for c in coins: if c <= a: dp[a] = min(dp[a], dp[a-c]+1)
return dp[amount] if dp[amount] != float('inf') else -1
```

Time Complexity: $O(\text{amount} \times \text{coins})$ **Space Complexity:** $O(\text{amount})$

■ Problem 19 (Medium): Activity Selection

Given n activities with start and finish times, select the maximum number of non-overlapping activities.
Activities: [(1,4),(3,5),(0,6),(5,7),(3,9),(5,9),(6,10),(8,11)] → Max = 4

■ Solution & Explanation

Greedy Strategy: Always choose the activity that **finishes earliest** (and doesn't overlap with the last selected). This leaves maximum room for future activities.

```
def activity_selection(activities): # Sort by finish time activities.sort(key=lambda x: x[1])
selected = [activities[0]] last_finish = activities[0][1]
for start, finish in activities[1:]: if start >= last_finish: # no overlap selected.append((start, finish))
last_finish = finish return selected, len(selected)
acts = [(1,4),(3,5),(0,6),(5,7),(3,9),(5,9),(6,10),(8,11)]
print(activity_selection(acts)[1]) # 4
```

Time Complexity:	$O(n \log n)$ — sorting	Space Complexity:	$O(1)$
-------------------------	---	--------------------------	--------------------------

CHAPTER
13

Advanced Topics

Trie, Sliding Window, Two Pointers & More

13.1 Trie (Prefix Tree)

A **Trie** is a tree data structure used for **string searching**. Each node represents a character. Efficient for prefix matching, autocomplete, and spell-check. Insert and Search are $O(L)$ where L = word length.

```
class TrieNode: def __init__(self): self.children = {} self.is_end = False # marks end of word
class Trie: def __init__(self): self.root = TrieNode()
def insert(self, word): node = self.root
for ch in word: if ch not in node.children: node.children[ch] = TrieNode()
node = node.children[ch]
node.is_end = True
def search(self, word): node = self.root
for ch in word: if ch not in node.children: return False
node = node.children[ch]
return node.is_end
def starts_with(self, prefix): node = self.root
for ch in prefix: if ch not in node.children: return False
node = node.children[ch]
return True
```

13.2 Sliding Window Technique

The **Sliding Window** pattern maintains a window (subarray/substring) that slides over data. Instead of recomputing the window from scratch each time, we **add the new element and remove the old one**. This turns $O(n^2)$ solutions into $O(n)$.

■ Problem 20 (Medium): Maximum Sum Subarray of size K

Find the maximum sum of any contiguous subarray of exactly size k . Example: `arr=[2,1,5,1,3,2]`, `k=3`
→ Output: 9 (subarray [5,1,3])

■ Solution & Explanation

Approach: Use a sliding window. Compute sum of first k elements. Then slide: add next element, remove first element of previous window.

```
def max_sum_subarray(arr, k): window_sum = sum(arr[:k]) # initial window
max_sum = window_sum
for i in range(k, len(arr)): window_sum += arr[i] - arr[i-k] # slide window
max_sum = max(max_sum, window_sum)
return max_sum
print(max_sum_subarray([2,1,5,1,3,2], 3)) # 9
```

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$

13.3 Two Pointer Technique

The **Two Pointer** technique uses two pointers (indices) to scan an array, often from both ends or at different speeds. Especially powerful on **sorted arrays**.

■ Problem 21 (Easy): Pair with Target Sum

Given a sorted array, find a pair that sums to target. Example: arr=[1,2,3,4,6], target=6 → Output: [1,3] (indices, 2+4=6)

■ Solution & Explanation

Approach: Start with left=0, right=n-1. If sum==target → found. If sum<target → move left right. If sum>target → move right left.

```
def pair_with_target_sum(arr, target): left, right = 0, len(arr)-1 while left < right:
    current_sum = arr[left] + arr[right] if current_sum == target: return [left, right]
    elif current_sum < target: left += 1 else: right -= 1 return []
print(pair_with_target_sum([1,2,3,4,6], 6)) # [1, 3]
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Quick Reference — Problems & Patterns

#	Problem	Pattern	Complexity
1	Two Sum	Hash Map	$O(n)$
2	Max Subarray	Kadane's	$O(n)$
3	Reverse Linked List	Two Pointers	$O(n)$
4	Detect Cycle	Floyd's	$O(n)$
5	Valid Parentheses	Stack	$O(n)$
6	Level Order BFS	Queue / BFS	$O(n)$
7	Max Depth Tree	DFS Recursion	$O(n)$
8	Lowest Common Ancestor	BST Property	$O(h)$
9	Kth Largest	Min-Heap	$O(n \log k)$
10	Valid Anagram	Hash Map / Counter	$O(n)$
11	Longest Consecutive	Hash Set	$O(n)$
12	Number of Islands	DFS Flood Fill	$O(m \times n)$
13	Sort Colors	3 Pointers	$O(n)$
14	Search Rotated Array	Binary Search	$O(\log n)$
15	Fibonacci	DP Memoization	$O(n)$
16	0-1 Knapsack	DP Tabulation	$O(n \times W)$
17	LCS	DP 2D Table	$O(m \times n)$
18	Coin Change	DP / Greedy	$O(A \times n)$
19	Activity Selection	Greedy	$O(n \log n)$
20	Max Sum Subarray (k)	Sliding Window	$O(n)$
21	Pair with Target Sum	Two Pointers	$O(n)$

■ **Tip:** Practice each pattern on at least 3 similar problems to truly master it. Start with Easy, then Medium, then Hard.

Data Structures & Algorithms — Fundamentals Guide

This guide covered 13 core topics and 21 problems from Easy to Hard. The patterns you've learned — Two Pointers, Sliding Window, BFS/DFS, DP, Greedy — appear in the vast majority of coding interviews and real-world software challenges.

Next steps: implement each data structure from scratch, solve 5 problems per pattern on platforms like LeetCode or HackerRank, and review time/space trade-offs regularly.